

P-Adic Linear Forms In Logarithms And Constraints On Modified Wall–Sun–Sun Primes In All Lucas Sequences: Data Book

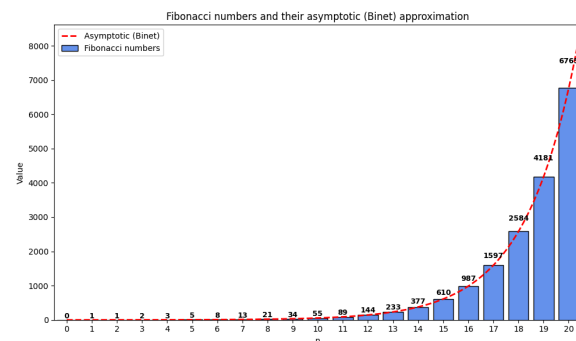
Owen Reich

All figures presented were created by the Finalist using Python 3.14 and corresponding Python and external dependences, all of which are cited.

1. Introduction

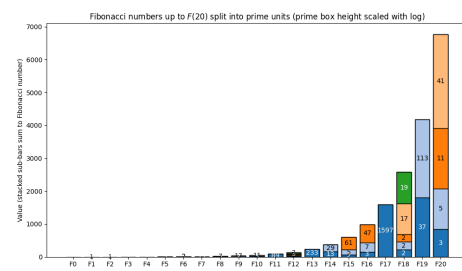
This is the accompanying data book for the ISEF project “*p*-Adic Linear Forms in Logarithms and Constraints on Modified Wall–Sun–Sun Primes in All Lucas Sequences” by Owen Reich. It includes:

- Computational (experimental) scripts in Python 3.14
 - Dependencies
 - Workflow descriptions
 - Recommended parameters/input
 - Sample output
- Retrieved experimental data
 - Datasets if manageably small
 - Links to datasets if very large
 - Figures



I did not keep a log of data collection including timestamps or original computer logs, so data is reconstructed after experimentation. This means that **data is reproduced from my actual experimentation, not original**. This data book also does not contain:

- Daily “research logs”
- Proof attempts and discarded versions
- Ideas
- Reflection or analysis



Due to the project being a mathematical project and every experiment is **100% deterministic**, the lack of detailed experimental reproduction steps does not lead to anomalies or lack of reproducibility.

Scripts are available at:

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/tree/main/scripts

And graphs and data are available at:

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/tree/main/graphs_and_data

Scripts are not in this document due to length, but are linked directly in each sub-section.

2. Python Scripts Linked and Descriptions

2.1 Computational Scripts

2.11 `lucas_universe_anomaly_search.py`

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/lucas_universe_anomaly_search

Overview

Systematically searches a parameter space of Lucas universes to find exceptional universes that maximize divisibility. The script uses matrix exponentiation modulo prime powers and ranks results by valuation and complexity, providing timed leaderboards.

Dependencies

ArgParse provides command-line argument parsing so the user can configure search range, valuation search depth, verbosity, and the number of reported results. Time is used for performance measurement. NumPy enables efficient matrix multiplication and modular reduction. SymPy gives fast prime generation and Legendre symbol evaluation.

Workflow Description

1. Helper routines give matrix exponentiation modulo powers of primes, calculating the p-adic valuation of the given Lucas term.
2. The script enumerates (1) over primes in a given range.
3. The script then enumerates (1) and (2) over all admissible P, Q pairs within a user defined box.
4. The script extracts the most extreme divisibility for ranking universes in a small leaderboard based on (in order of importance): largest valuation achieved, and then smallest $|P| + |Q|$ for tiebreaking.
5. Finally, the script reports time statistics and prints the highest ranking universes.

Example Input

```
python anomaly_universes_finder.py -b 50 -p 200 --max_k 4 --top_n 3
```

Example Output

```
Primes: 45 (3..199)
Universes to scan: 2545 (P in 1..50, Q in -25..25)
max_k (valuation depth) = 4
Starting scan...
  scanned 1000/2545 universes -- time 2.0s
  scanned 2000/2545 universes -- time 3.9s
  scanned 2545/2545 universes -- time 5.0s

Scan complete.
Time elapsed: 4.95 seconds
Universes checked: 2545
Top universes found (value, |P|+|Q|, P, Q):
1. max= 4, complexity= 2, P= 1, Q= 1
2. max= 4, complexity= 4, P= 2, Q= 2
3. max= 4, complexity= 6, P= 2, Q= 4
```

2.12 prime_valuation_classification_scan.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/prime_valuation_classification_scan.py

Overview

A high throughput scanner that classifies primes in specific Lucas sequences, reporting counts of Wall–Sun–Sun primes. It is designed for large scale empirical searches.

Dependencies

Math is used for logarithmic estimates and general numeric computations. Time measures computation time and progress reporting for long runs. Typing provides annotations that clarify function inputs and outputs.

Workflow Description

1. The script uses a sieve routine to generate the first n primes.
2. Efficient modular matrix exponentiation computes Lucas terms mod p and mod p^2 with the Legendre symbol.
3. For each prime p the script determines if p is a regular prime or a Wall–Sun–Sun prime and aggregates the number of each valuation class.

Example Input

```
python pell_wss_calculator.py
Lucas sequence valuation experiment
U_n = P*U_{n-1} - Q*U_{n-2}
Examples:
  Pell:      P=2, Q=-1
  Fibonacci: P=1, Q=-1

Enter P: 1
Enter Q: -1
```

Example Output

```
Processed 100,000/1,000,000 primes in 1.5s, rates: [1, 100000, 0]
Processed 200,000/1,000,000 primes in 3.2s, rates: [1, 200000, 0]
Processed 300,000/1,000,000 primes in 4.9s, rates: [1, 300000, 0]
Processed 400,000/1,000,000 primes in 6.7s, rates: [1, 400000, 0]
Processed 500,000/1,000,000 primes in 8.6s, rates: [1, 500000, 0]
Processed 600,000/1,000,000 primes in 10.6s, rates: [1, 600000, 0]
Processed 700,000/1,000,000 primes in 12.5s, rates: [1, 700000, 0]
Processed 800,000/1,000,000 primes in 14.5s, rates: [1, 800000, 0]
Processed 900,000/1,000,000 primes in 16.5s, rates: [1, 900000, 0]
Done: processed 1,000,000 primes in 18.54s

999999 primes divide U_n exactly once.
0 primes divide U_n more than once.

Primes dividing U_n with valuation ≥ 2:
[]
```

2.13 single_universe_valuation_search.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/single_universe_valuation_probe.py

Overview

A compact testing script implementing a single-universe p-adic valuation check for one Lucas sequence. It computes sequences via recursion. It is intended as a test concept rather than a full utility.

Dependencies

Math provides integer square root computation and basic arithmetic utilities.

Workflow Description

1. The script evaluates p -adic valuations of Lucas sequence terms indexed by the Legendre symbol. First, the Lucas sequence is computed via integer recursion.
2. Next, for each prime p , the script evaluates $v_p(U_{p-(p/D)}(P, Q))$ for user input P, Q .
3. **No output is generated** - this script is more of a test concept of a single function than a full utility, however the function was not used elsewhere in the project.
4. The script is essentially (2.04) but for one universe and without graphing.

2.2 Experimental Scripts

2.21 euler_lucas_conjecture_tester.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/euler_lucas_conjecture_tester.py

Overview

Numerically checks a formulated conjecture linking Euler's sum of powers conjecture to high divisibility Lucas terms. It iterates over proposed counterexamples and parameter pairs.

Dependencies

Typing provides clarified annotations. *Math* gives fundamental arithmetic utilities for modular exponentiation.

Workflow Description

1. The script defines mathematical helper functions for the Legendre symbol and modular matrix exponentiation.
2. For each Euler counterexample and Lucas parameters the script checks prime divisibility of the associated Lucas term and an approximate p -adic valuation is also estimated.
3. Multiple counterexamples and multiple Lucas pairs are iterated, and results are printed in the terminal

Example Input

```
python l-Euler Hypothesis Tester.py
```

Example Output

```
example a=[27, 84, 110, 133] = 144^5 with exponent p=5, testing
(P,Q)=(1,-1):
D=5, leg(D/p)=0, m=5
U_m mod p^2 = 5, p^2 | U_m? False
approx p-adic valuation of U_m (capped): 1
```

```
example a=[27, 84, 110, 133] = 144^5 with exponent p=5, testing  
(P,Q)=(1,1):
```

```
  D=-3, leg(D/p)=-1, m=6  
  U_m mod p^2 = 0, p^2 | U_m? True  
  approx p-adic valuation of U_m (capped): 0
```

```
example a=[27, 84, 110, 133] = 144^5 with exponent p=5, testing  
(P,Q)=(2,-1):
```

```
  D=8, leg(D/p)=-1, m=6  
  U_m mod p^2 = 20, p^2 | U_m? False  
  approx p-adic valuation of U_m (capped): 1
```

```
example a=[27, 84, 110, 133] = 144^5 with exponent p=5, testing  
(P,Q)=(3,-1):
```

```
  D=13, leg(D/p)=-1, m=6  
  U_m mod p^2 = 10, p^2 | U_m? False  
  approx p-adic valuation of U_m (capped): 1
```

```
example a=[85282, 28969, 3183, 55] = 85359^5 with exponent p=5, testing  
(P,Q)=(1,-1):
```

```
  D=5, leg(D/p)=0, m=5  
  U_m mod p^2 = 5, p^2 | U_m? False  
  approx p-adic valuation of U_m (capped): 1
```

```
example a=[85282, 28969, 3183, 55] = 85359^5 with exponent p=5, testing  
(P,Q)=(1,1):
```

```
  D=-3, leg(D/p)=-1, m=6  
  U_m mod p^2 = 0, p^2 | U_m? True  
  approx p-adic valuation of U_m (capped): 0
```

```
example a=[85282, 28969, 3183, 55] = 85359^5 with exponent p=5, testing  
(P,Q)=(2,-1):
```

```
  D=8, leg(D/p)=-1, m=6  
  U_m mod p^2 = 20, p^2 | U_m? False  
  approx p-adic valuation of U_m (capped): 1
```

```
example a=[85282, 28969, 3183, 55] = 85359^5 with exponent p=5, testing  
(P,Q)=(3,-1):
```

```
  D=13, leg(D/p)=-1, m=6  
  U_m mod p^2 = 10, p^2 | U_m? False  
  approx p-adic valuation of U_m (capped): 1
```

A Note on the Conjecture Presented Here

The conjecture that this script tests is probably false, as shown above, and it reflects my attempt to generalize from Wall–Sun–Suns and Fermat’s Last Theorem into generalized Wall–Sun–Suns and Euler’s Sum of Powers conjecture. While a connection may exist, it will most likely not come from guessing and instead a careful, rigorous theoretical proof. The conjecture:

$$\sum_{n=1}^k a_n^p = b^p \Rightarrow p^k \mid U_{p-(p|D)}(P, Q)$$

With coefficients P and Q dependent on $a_0 \dots a_n$. While the above output does not disprove the conjecture it shows that P and Q would have to depend somehow on the formulation of the equality.

2.22 fibonacci_digit_prng.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/fibonacci_digit_prng.py

Overview

Simple, deterministic, pseudorandom digit generator based on the Fibonacci recurrence. More of a simple experiment than a production-grade script.

Dependencies

Sys is used to read optional command-line arguments. No other libraries are used.

Workflow Description

1. Fixed starting parameters $j = 3$ and $k = 7$ and a default modulus of 10 and a default seed string are defined. Command line arguments may override them.
2. The seed string is converted into a list of integer digits.
3. The program checks that the seed is long enough.
4. For 20 iterations:
 - a. A new digit is computed as the sum of the digits at position j and k .
 - b. The digit list is shifted left, discarding the first element.
 - c. The new digit is appended to the end.
 - d. The generated value is stored
5. The list of generated digits is printed to the terminal.

Example Input

```
python random_number_generator_fibonacci.py 1938482 9
```

Example Output

```
[5, 4, 8, 7, 0, 5, 0, 8, 6, 6, 2, 2, 1, 7, 4, 6, 8, 0, 7, 2]
```

2.23 `lucas_divisibility_heuristic_test.py`

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/lucas_divisibility_heuristic_test.py

Overview

Implements an empirical heuristic test comparing predicted counts from a loose bound against observed p-adic divisibility statistics for a Lucas universe. The script exports values to help assess the heuristic's accuracy.

Dependencies

Math gives square roots, logarithms, and complex numbers for complex characteristic roots.

Matplotlib provides plotting and exportation.

Workflow Description

1. The user gives Lucas parameters with an interactive terminal interface.
2. The script generates primes and evaluates the divisibility of the terms by p^k .
3. The script accumulates a heuristic expectation if $9600p^2$ is treated as an accurate prediction and not a bound
4. Matplotlib produces the visualization.

Example Input

```
Lucas universe expectation vs actual (primes up to 200).
Enter integer P (e.g. 1 for Fibonacci-like): 1
Enter integer Q (e.g. -1 for Fibonacci-like): -1
```

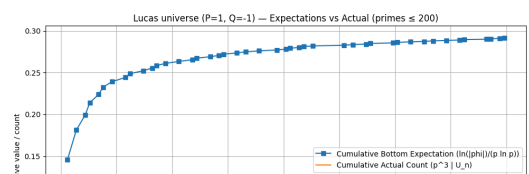
Example Output

```
Parameters: P=1, Q=-1, D = 5
(Primes dividing D will be skipped.)
Plot saved to: lucas_expectations_P1_Q-1.png
```

Summary:

```
Number of primes considered: 44
Top expectation total (sum 9600/(p^2)): 5694.818082
Bottom expectation total (sum ln(|phi|)/(p ln p)): 0.291423
Actual count of primes with p^3 | U_n: 0
```

Done.



Note on Bound Presented Here

The bound presented here, $9600p^2$, is a version of the $2400p^2$ bound presented in my final project. This worse bound comes from a worse optimization of constants in the application of Bugeaud–Laurent.

2.24 nbonacci_modular_period_explorer.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/nbonacci_modular_period_explorer.py

Overview

Minimal, dependency-free program that computes and reduces terms of higher-order linear recurrences like the Fibonacci sequence. It is not designed for high-output research or computation.

Dependencies

The script only uses core Python functionality with no dependencies.

Workflow Description

1. The script defines a recursive formula for the n -bonacci sequence, $f(k) = f(k-1) + f(k-2) \dots + f(k-n)$.
2. The program prompts for an integer modulus.
3. For indices 1 to 199 the script calculates the recursive sequence value, reduces the result modul the user input, and prints the raw and reduced value.

Example Input

```
python period of n-bonacci.py
period? 10
```

Example Output

```
period? 10
0, 0
1, 1
1, 1
2, 2
4, 4
7, 7
13, 3
24, 4
44, 4
81, 1
149, 9
...
```

2.3 Graphing Scripts

2.31 fibonacci_legendre_quotient_plotter.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/fibonacci_legendre_quotient_plotter.py

Overview

Computes rational values of the Fibonacci quotient at Legendre offset indices. It combines exact integer arithmetic with sieving and plotting, allowing the high-precision measurement of quotient growth and exceptional Wall–Sun–Sun primes.

Dependencies

Argparse provides CLI functionality. *Fractions* enables rational arithmetic to avoid precision loss due to floating errors. *Matplotlib* gives figure creation and exporting. *Typing* supports code annotation.

Workflow Description

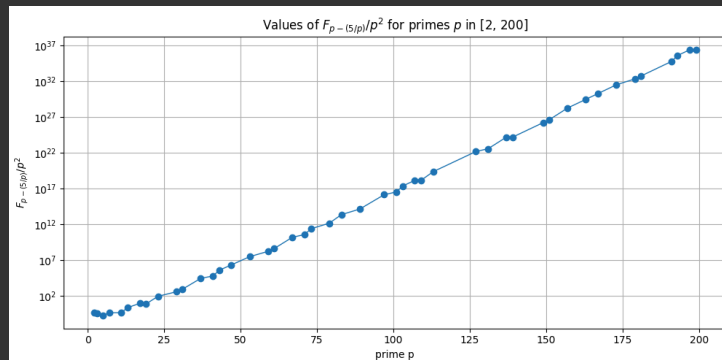
1. First, the script takes CLI inputs from the user.
2. Then, the script generates primes in a range with sieving.
3. Next, script computes the exact Fibonacci quotient using matrix exponentiation for Fibonacci numbers and calculates the Legendre symbol with pure Python.
4. Finally, the script uses *matplotlib* to create a graph of the quotient, as well as printing a tabulated output to the terminal.

Example Input

```
python fibonacci_quotient.py --start 2 --end 200 --log --save output.png
```

Example Output

```
p    -> Fp-(5/p) / p2 (exact)    -> (float plotted)
2    -> 1/2    -> 0.5
3    -> 1/3    -> 0.333333
5    -> 1/5    -> 0.2
7    -> 3/7    -> 0.428571
11   -> 5/11   -> 0.454545
13   -> 29/13  -> 2.23077
17   -> 152/17 -> 8.94118
19   -> 136/19 -> 7.15789
23   -> 2016/23 -> 87.6522
29   -> 10959/29 -> 377.897
31   -> 26840/31 -> 865.806
37   -> 1056437/37 -> 28552.4
41   -> 2495955/41 -> 60877
```



```
43 -> 16311831/43 -> 379345
47 -> 102287808/47 -> 2.17634e+06
53 -> 1627690024/53 -> 3.07111e+07
59 -> 10021808981/59 -> 1.69861e+08
61 -> 25377192720/61 -> 4.1602e+08
67 -> 1085424779823/67 -> 1.62004e+10
71 -> 2681584376185/71 -> 3.77688e+10
73 -> 17876295136009/73 -> 2.44881e+11
79 -> 113220181313816/79 -> 1.43317e+12
83 -> 1933742696582736/83 -> 2.32981e+13
89 -> 12360536835574179/89 -> 1.38882e+14
97 -> 1394864457161925217/97 -> 1.438e+16
101 -> 3507176714646157575/101 -> 3.47245e+16
103 -> 23571778916504612451/103 -> 2.28852e+17
107 -> 155523623837575361328/107 -> 1.45349e+18
109 -> 152669979363491409744/109 -> 1.40064e+18
113 -> 2642576343530770503704/113 -> 2.33856e+19
127 -> 1982116737665744001184443/127 -> 1.56072e+22
131 -> 5030798638073511770858765/131 -> 3.8403e+22
137 -> 225989772612884036918821112/137 -> 1.64956e+24
139 -> 222738121208382108330061096/139 -> 1.60243e+24
149 -> 25556388788416277561281235799/149 -> 1.71519e+26
151 -> 66021302497942406531221230200/151 -> 4.37227e+26
157 -> 2983063546028389514347409745197/157 -> 1.90004e+28
163 -> 51558514312724753881863605072271/163 -> 3.1631e+29
167 -> 344922947517513508300635920271648/167 -> 2.06541e+30
173 -> 5974730193226528891482448529250184/173 -> 3.4536e+31
179 -> 39578737005477757077372122318267741/179 -> 2.2111e+32
181 -> 102473523146254067470663748737014960/181 -> 5.66152e+32
191 -> 11943545766832657047793986402399015745/191 -> 6.25317e+34
193 -> 81013966736622771557406030001128754369/193 -> 4.19761e+35
197 -> 544003308729504197349882716389171244392/197 -> 2.76144e+36
199 -> 538535938792524255667974347380234849976/199 -> 2.70621e+36
Saved plot to output.png
```

2.32 fibonacci_structure_visualizer.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/fibonacci_structure_visualizer.py

Overview

An interactive plotting utility offering visualizations of the Fibonacci sequence. The script is intended for intuitive illustration and data visualization rather than raw computation.

Dependencies

Math gives basic operations for calculating Binet's formula. NumPy provides continuous input for the continuous Binet's formula approximation. SymPy gives fast integer factorization to display prime factors of Fibonacci numbers. Matplotlib provides the graphing functionality of the script.

Workflow Description

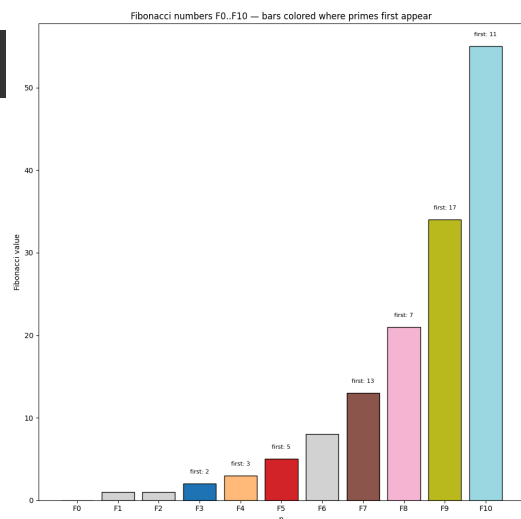
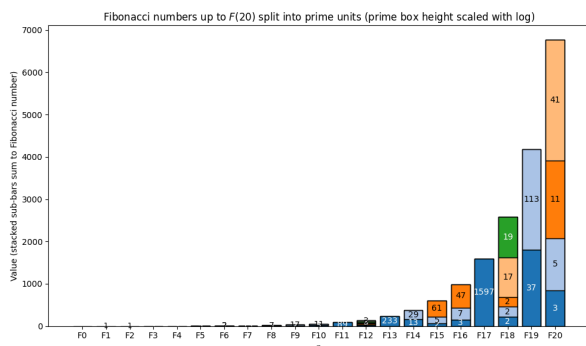
- The script has four different graphing options of the Fibonacci sequence:
 - A simple Fibonacci bar graph with an asymptotic approximation given by Binet
 - A Fibonacci bar graph with each bar where a prime appears as a factor for the first time in the sequence highlighted and labeled with the prime
 - A Fibonacci bar graph broken up into prime factors, scaled logarithmically
 - A log-scaled Fibonacci bar graph with the prime p appearing in $p^2 \mid F_{p-(p|5)}$
- The script goes item by item and asks you which functions to call and what to graph to. No CLI is available, only interactive terminal inputs.

Example Input

```
Would you like to graph the Fibonacci sequence with the Binet
approximation? [y/n] n
Moving to option 2:
Would you like to graph the Fibonacci sequence with prime factors shown?
[y/n] y
To what index? 20
Would you like to graph the Fibonacci sequence with prime first traces
shown? [y/n] y
To what index? 10
Would you like to graph the Fibonacci sequence with primes shown appearing
in  $F_{p-(p|5)}$ ? [y/n] n
```

Example Output

Your graphs are done.



2.33 lucas_prime_bound_visualizer.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/lucas_prime_bound_visualizer

Overview

Implements a fast computation to calculate and graph the divisibility bound split across Legendre cases. It uses discrete visualization to show how the theoretical bound behaves.

Dependencies

Matplotlib is used for graphing the scatter plot of the divisibility bound across primes. NumPy is for numerical efficiency although not necessary.

Workflow Description

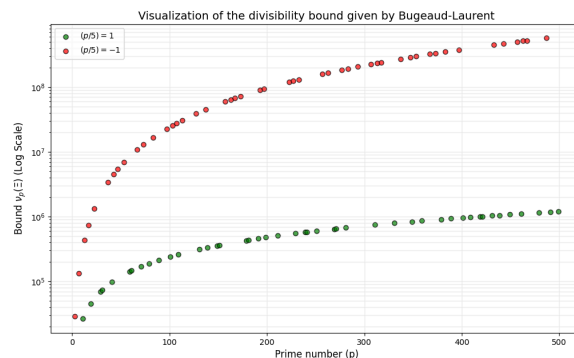
1. Implements a Legendre symbol calculator to split the bound
2. Defines the formula $= 2400p^f$ based on the Legendre symbol
3. Basic primality test to find small primes to calculate the bound
4. Graphs the bound discretely across primes
5. Prints the first several primes, their associated Legendre values, and computed bounds

Example Input

```
python script.py
```

Example Output

Prime	Legendre	Bound
3	-1	...
7	-1	...
11	1	...
13	-1	...
...		



2.34 lucas_valuation_bound_overlay.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/lucas_valuation_bound_overlay.py

Overview

Constructs an overlay of p-adic valuations for many Lucas universes and visualizes them against the theoretical bound. The tool automates universe generation and valuation computation. It plots each universe at once to support comparative analysis.

Dependencies

NumPy provides matrix exponentiation and modular reduction as well as storage of valuation data. Matplotlib is used to construct the overlay of p-adic valuations across multiple Lucas universes with

logarithmic scaling and anomaly highlighting. SymPy gives prime enumeration and Legendre symbol calculation. Math supports auxiliary numerical operations.

Workflow Description

1. First, the user specifies parameter ranges, and the script constructs all admissible universes and removes degenerate universes.
2. Next, for each remaining universe and each prime the script computes the p-adic valuation of a designated Lucas sequence term using NumPy modular matrix exponentiation.
3. After processing all universes, the program identifies the largest valuation universe and the Fibonacci universe
4. Finally, it produces a logarithmic overlay with matplotlib showing valuations across universes, marking anomalous primes, and draws the linear $2400p$ bound.

Example Input

```
python bound_divisibility_isef.py
```

```
--- Lucas Universe Overlay (skip periodic +  $P^2 = kQ$  for  $k=0..3$ ; omit
zeros) ---
Enter P_min (e.g. 1): 1
Enter P_max ( $\geq P_{\min}$ , e.g. 5): 4
Enter max  $|Q|$  (positive integer, e.g. 5): 3
Enter Prime Bound (e.g. 200): 100
```

Example Output

```
Requested universes: 24; skipping 4 universes (periodic or  $P^2=kQ$ ).
Skipped examples (up to 40): [(1, 1), (2, 1), (2, 2), (3, 3)]
Scanning 20 universes; primes up to 100...
```

2.35 parallel_lucas_universe_overlay.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/parallel_lucas_universe_overlay.py

Overview

Parallelized implementation of multiple Lucas universe scanning that aggregates divisibility data into a ranked overlay plot. It is functionally equivalent to (2.04).

Dependencies

Math, sys, time, os, and datetime all support numerical operations, timing, filesystem access and timestamped data exportation. Multiprocessing enables parallel computation across CPU cores to accelerate scanning of many parameter pairs. Functools gives various utilities though not essential.

NumPy provides efficient array storage and numerical handling. Matplotlib generates the final visualizations of valuations across primes. SymPy gives prime generation and Legendre computation.

Workflow Description

1. User input parameters for P and Q and prime ranges are given. Degenerate universes are filtered out to retain only meaningful Lucas universes.
2. For each valid P, Q pair, worker processes iterate over primes:
 - a. The index is calculated with the helper Legendre function.
 - b. U_n is calculated by fast matrix exponentiation modulo p .
 - c. Valuations and anomalies are recorded.
3. Results from all workers are combined and the script identifies the max achieved valuation.
4. A log plot with Matplotlib displays valuations versus the universal bound, marking anomalies.

Example Input

```
python universe_valuation_overlay.py
Lucas Universe Overlay
Enter P_min: 1
Enter P_max (>= P_min): 20
Enter max |Q|: 10
Enter Prime Bound: 200
```

Example Output

```
Scanning 393 universes; primes up to 200...
utilizing 10 CPU cores.
Progress: [=====] 100% | 393/393 | 507.5 u/s
Scan complete.
Universe with max_v=3: P=5, Q=-1
Universe with max_v=3: P=5, Q=3
Universe with max_v=3: P=8, Q=5
Universe with max_v=3: P=10, Q=-3
Universe with max_v=3: P=9, Q=-4
Universe with max_v=3: P=10, Q=-4
Universe with max_v=3: P=12, Q=-10
Universe with max_v=3: P=11, Q=-7
Universe with max_v=3: P=9, Q=3
Universe with max_v=3: P=11, Q=-4
Universe with max_v=3: P=11, Q=-2
Universe with max_v=3: P=13, Q=-10
Universe with max_v=3: P=14, Q=-10
Universe with max_v=3: P=14, Q=6
Universe with max_v=3: P=16, Q=2
```

```
Universe with max_v=3: P=16, Q=3
Universe with max_v=3: P=16, Q=-7
Universe with max_v=3: P=16, Q=6
Universe with max_v=3: P=16, Q=9
Universe with max_v=3: P=17, Q=-4
Universe with max_v=3: P=19, Q=5
Universe with max_v=3: P=19, Q=6
Universe with max_v=3: P=19, Q=-7
Universe with max_v=3: P=19, Q=9
Generating plot...
Writing data to exported_data/all_universes_optimized.csv...
Done.
```

2.36 valuation_heatmap_renderer.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/valuation_heatmap_renderer.py

Overview

Generates a smooth color-mapped heatmap for numerical matrices using Gaussian smoothing and custom color maps. It is primarily a visualization helper for intuitive reports.

Dependencies

NumPy gives multidimensional array storage, NaN to All conversion, and distance calculations for smooth color gradients. Matplotlib gives custom continuous color mapping. Scipy optionally provides Gaussian smoothing for better interpolation; replaced by NumPy if not present.

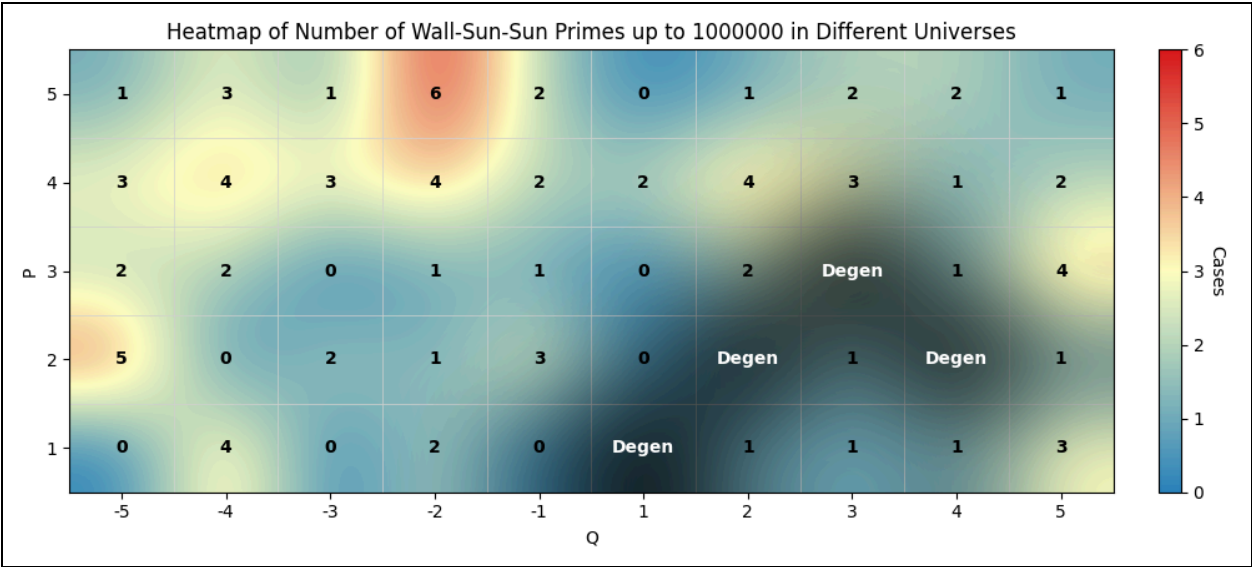
Workflow Description

1. The script is given the data matrix (calculated from `pell_wss_calculator.py` at (2.12)).
2. NumPy/SciPy gives Gaussian smoothing and creates a continuous color overlay with Matplotlib.
3. The script adds annotation to the color map for a finished product.

Example Input

No user input, values are hardcoded from (2.12) although a possible future extension would be to merge these scripts to create larger heatmaps or with higher divisibility primes instead of WSSs.

Example Output



2.4 Report Generation Scripts

2.41 lucas_universe_profile_pdf.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/lucas_universe_profile_pdf

Overview

Produces a single page PDF profiling a Lucas universe for p-adic valuations and compiles statistical summaries. It integrates a formatted matplotlib graphic output for visualization.

Dependencies

SymPy provides prime enumeration and Legendre symbol calculation. Matplotlib is used to construct a fully formatted visualization and export it to a single-page PDF. Tdqm is optionally used for long prime scans. Argparse is, math, time, and textwrap are used to support command-line configuration, numerical calculations, performance measurement, and nicely formatted PDF text.

Workflow Description

1. Helper routines implement matrix multiplication and exponentiation to compute exact Lucas sequence values and detect periodic universes.
2. The script evaluates p-adic valuations of the sequence for primes below a bound and identifies “anomalies”, which are when the valuation exceeds 1.
3. Next, it compiles statistical data including the maximum valuation “staircase”, anomaly counts, and a comparison against the $2400p^2$ bound.
4. Finally, the script uses Matplotlib to render the PDF page, and a short terminal summary is printed.

Example Input

Terminal:

```
python binder_maker_108_universes.py --P 3 --Q -4 --prime_limit 100000
--max_k 6
```

Interactive usage:

```
python binder_maker_108_universes.py
Enter integer P: 3
Enter integer Q: -4
```

Example Output

```
Computing primes: 100%|██████████| 9591/9591 [00:00<00:00, 27824.17it/s]
Saved 'Profile_P3_Q-4_final2.pdf' -- primes: 9,591, anomalies: 2, seq-zero
primes: 0. Elapsed 0.5s.
```

2.42 wss_composite_statistical_report.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/wss_composite_statistical_report.py

Overview

Large-scale composite analysis that scans integers for Wall–Sun–Sun criteria and writes an HTML report with visualizations. The script is mainly statistically focused rather than theoretical.

Dependencies

Argparse, base64, csv, io, math, os, random, sys, time, collections, datetime, heapq, typing provide command-line parsing, different base encoding, file handling, numerical utilities, timing, data structures, and type annotations. Numpy gives efficient numerical arrays and spectral analysis. Matplotlib generates plots embedded into the final HTML report. Psutil optionally enables runtime memory-usage reporting. SciPy supplies statistical hypothesis tests such as chi-square.

Workflow Description

1. A segmented sieve finds all composite numbers up to a specified bound.
2. For each n the script computes the quotient from the Fibonacci term with the Legendre symbol offset.
3. The script verified Wall–Sun–Sun composites with $c^2 \mid F_{c-(c|D)}$ and aggregates them.
4. While iterating, the script maintains histogram counts, modular residue stats, detection of anomalous composites, and other statistical properties.
5. Using the sampled data, the program produces histograms, Q-Q plots, scatter plots, Fourier spectral analysis, and goodness of fit metrics.
6. Results are written to an HTML report with embedded images and an optional CSV file containing sampled data and Wall–Sun–Sun composite lists.

Example Input

```
python composite_wss_stream.py -bound 100000
```

Example Output

```
Output HTML: report-02-17-26-100000-composites.html
[10,000] composites processed -- 309708.2 comp/s, 3.2 s per million
[20,000] composites processed -- 328512.9 comp/s, 3.0 s per million
[30,000] composites processed -- 329433.7 comp/s, 3.0 s per million
[40,000] composites processed -- 309010.3 comp/s, 3.2 s per million
[50,000] composites processed -- 289576.1 comp/s, 3.5 s per million
[60,000] composites processed -- 272345.2 comp/s, 3.7 s per million
[70,000] composites processed -- 268111.6 comp/s, 3.7 s per million
[80,000] composites processed -- 262319.2 comp/s, 3.8 s per million
```

```
[90,000] composites processed -- 256471.5 comp/s, 3.9 s per million
HTML report written: report-02-17-26-100000-composites.html
Done. Processed 90,407 composite numbers in 0.353s -- 256163.2 comp/s, 3.9s
per million.
```

2.43 wss_prime_statistical_report.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/wss_prime_statistical_report.py

Overview

Prime focused analogue to the composite scanner of (2.16).

Dependencies

Argparse provides command-line argument parsing for configuring execution parameters. Base64, io, and csv support in-memory images and optional exportation of sampled data. Math, random, time, and datetime give numerical operations, random sampling, timing, and timestamps. Collections and heapq enable efficient aggregation of anomalies during streaming. NumPy and matplotlib give numerical array processing and visualization for the HTML report. Psutil optionally enables runtime memory reporting. SciPy provides statistical hypothesis tests and goodness of fit.

Workflow Description

1. The script generates primes up to a bound using a segmented sieve.
2. Then, the script calculates the Fibonacci values offset by the Legendre symbol mod p .
3. It enumerates (2) across all generated primes and compiles statistics.
4. Next, the script uses matplotlib and SciPy to generate graphs/analysis to an HTML report.

Example Input

```
python script.py --bound 1000000 --sample-size 50000 --bins 40 --html
report.html --csv sample.csv
```

Example Output

```
Output HTML: report.html
[10,000] primes processed -- 117103.0 p/s, 8.5 s per million
[20,000] primes processed -- 141273.2 p/s, 7.1 s per million
[30,000] primes processed -- 150924.1 p/s, 6.6 s per million
[40,000] primes processed -- 155414.4 p/s, 6.4 s per million
[50,000] primes processed -- 157417.4 p/s, 6.4 s per million
[60,000] primes processed -- 156005.1 p/s, 6.4 s per million
[70,000] primes processed -- 154417.0 p/s, 6.5 s per million
HTML report written: report.html
Done. Processed 78,498 primes in 0.511s -- 153567.0 p/s, 6.5s per million.
```

2.5 “Wall–Sun–Sun Composite” Scripts

2.51 wss_composite_family_verifier.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/wss_composite_family_verifier

Overview

Targets two structured families of integers to test and verify Wall–Sun–Sun composite behavior. Verified composites are reported with indices and timing statistics and can be exported to CSV for analysis.

Dependencies

Argparse enables command-line operation. Csv, os provide .csv file output and filesystem handling for recording results. Math, random, time, sys support numerical utilities, probabilistic selection, runtime measurement, and controlled program termination.

Workflow Description

1. The script searches for “Wall–Sun–Sun Composites” with two families $m = 2^{4k+3} + 1$ and $m = 2^n - 1$ with $n \bmod 12 \in \{2, 3, 6, 10, 11\}$. Bounds are set by the user.
2. Each candidate m undergoes Miller-Rabin probable prime testing. Values that appear prime or exceed the bit-length threshold are skipped.
3. For composite candidates, the script computes a Fibonacci index based on the Jacobi symbol.
4. The script evaluates $F_{\text{index}} \pmod{m^2}$ using fast-doubling formulas and checks if $F_{\text{index}} \equiv 0 \pmod{m^2}$. Successful cases are recorded as WSS composites.
5. Verified results are printed to the terminal and can be written to CSV optionally, and summarized separately for each family with timing statistics.

Example Input

```
Enter x (upper bound for k in family A; k runs 0..x): 5
Enter y (upper bound for n in family B; n runs 2..y): 20
```

Example Output

```
Scanning family A: m = 2^(4k+3)+1 for k=0..5
[A k=0] VERIFIED WSS composite: m=9 (2^3+1), idx=8
[A k=1] VERIFIED WSS composite: m=129 (2^7+1), idx=128
[A k=2] VERIFIED WSS composite: m=2049 (2^11+1), idx=2048
[A k=3] VERIFIED WSS composite: m=32769 (2^15+1), idx=32768
[A k=4] VERIFIED WSS composite: m=524289 (2^19+1), idx=524288
[A k=5] VERIFIED WSS composite: m=8388609 (2^23+1), idx=8388608
Family A done. Verified 6 entries; scanned k=0..5 in 0.00s
```

```
Scanning family B: m = 2^n - 1 for n <= 20, n mod 12 in {2,3,6,10,11}
[B n=6] VERIFIED WSS composite: m=2^6-1 (n=6), idx=64
[B n=10] VERIFIED WSS composite: m=2^10-1 (n=10), idx=1024
[B n=11] VERIFIED WSS composite: m=2^11-1 (n=11), idx=2048
[B n=14] VERIFIED WSS composite: m=2^14-1 (n=14), idx=16384
[B n=15] VERIFIED WSS composite: m=2^15-1 (n=15), idx=32768
[B n=18] VERIFIED WSS composite: m=2^18-1 (n=18), idx=262144
Family B done. Verified 6 entries; scanned n up to 20 in 0.00s
```

```
=== Summary ===
```

```
Family A: verified WSS composites (k, exp, m): 6 found
```

```
  k=0, exp=3, m(bitlen=4)
  k=1, exp=7, m(bitlen=8)
  k=2, exp=11, m(bitlen=12)
  k=3, exp=15, m(bitlen=16)
  k=4, exp=19, m(bitlen=20)
  k=5, exp=23, m(bitlen=24)
```

```
Family B: verified WSS composites (n, m): 6 found
```

```
  n=6, m(bitlen=6)
  n=10, m(bitlen=10)
  n=11, m(bitlen=11)
  n=14, m(bitlen=14)
  n=15, m(bitlen=15)
  n=18, m(bitlen=18)
```

```
(Entries labelled 'probable-prime' were skipped by the probable-prime test;
very large entries may be skipped due to max bit-length.)
```

2.52 wss_composite_statistical_report.py

https://github.com/Owen-Reich-Likes-Math/wss_primes_isef/blob/main/scripts/wss_composite_statistical_report.py

Overview

Large-scale composite analysis that scans integers for Wall–Sun–Sun criteria and writes an HTML report with visualizations. The script is mainly statistically focused rather than theoretical.

Dependencies

Argparse, base64, csv, io, math, os, random, sys, time, collections, datetime, heapq, typing provide command-line parsing, different base encoding, file handling, numerical utilities, timing, data structures, and type annotations. Numpy gives efficient numerical arrays and spectral analysis.

Matplotlib generates plots embedded into the final HTML report. Psutil optionally enables runtime memory-usage reporting. SciPy supplies statistical hypothesis tests such as chi-square.

Workflow Description

1. A segmented sieve finds all composite numbers up to a specified bound.
2. For each n the script computes the quotient from the Fibonacci term with the Legendre symbol offset.
3. The script verified Wall–Sun–Sun composites with $c^2 \mid F_{c-(c/D)}$ and aggregates them.
4. While iterating, the script maintains histogram counts, modular residue stats, detection of anomalous composites, and other statistical properties.
5. Using the sampled data, the program produces histograms, Q-Q plots, scatter plots, Fourier spectral analysis, and goodness of fit metrics.
6. Results are written to an HTML report with embedded images and an optional CSV file containing sampled data and Wall–Sun–Sun composite lists.

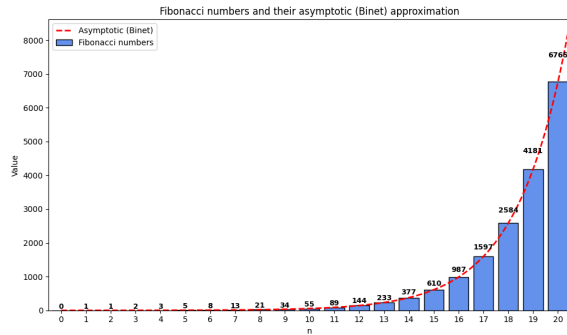
Example Input

```
python composite_wss_stream.py -bound 100000
```

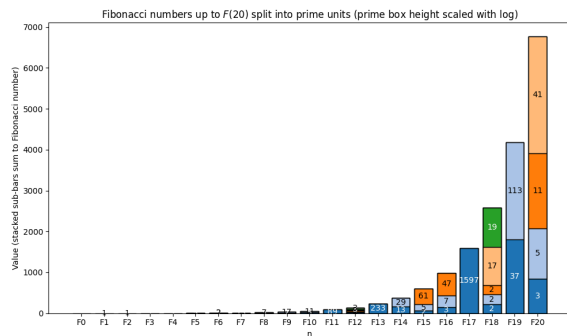
Example Output

```
Output HTML: report-02-17-26-100000-composites.html
[10,000] composites processed -- 309708.2 comp/s, 3.2 s per million
[20,000] composites processed -- 328512.9 comp/s, 3.0 s per million
[30,000] composites processed -- 329433.7 comp/s, 3.0 s per million
[40,000] composites processed -- 309010.3 comp/s, 3.2 s per million
[50,000] composites processed -- 289576.1 comp/s, 3.5 s per million
[60,000] composites processed -- 272345.2 comp/s, 3.7 s per million
[70,000] composites processed -- 268111.6 comp/s, 3.7 s per million
[80,000] composites processed -- 262319.2 comp/s, 3.8 s per million
[90,000] composites processed -- 256471.5 comp/s, 3.9 s per million
HTML report written: report-02-17-26-100000-composites.html
Done. Processed 90,407 composite numbers in 0.353s -- 256163.2 comp/s, 3.9s
per million.
```

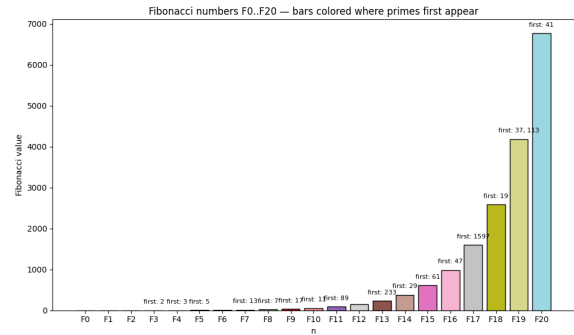
3. Graphs and Data With Python Script Sources



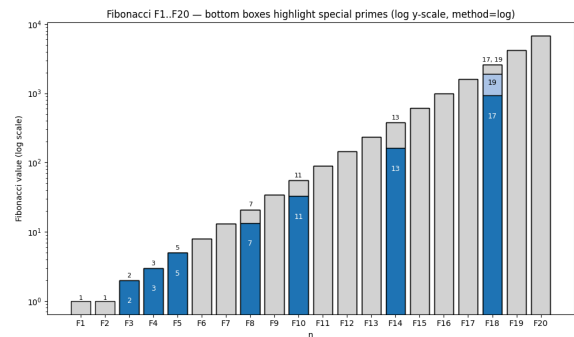
➤ **Fig 2.32.1.** The Fibonacci sequence in discrete integer values, with an asymptotic approximation given from Binet's formula.



➤ **Fig 2.32.2.** The Fibonacci sequence in discrete integer values with each bar broken into prime factors, scaled logarithmically.



➤ **Fig 2.32.3.** The Fibonacci sequence, with first traces of primes shown. A first trace is the first Fibonacci number with a certain prime dividing it. A classical theorem is that the first trace index always divides the index $p - (p|5)$.

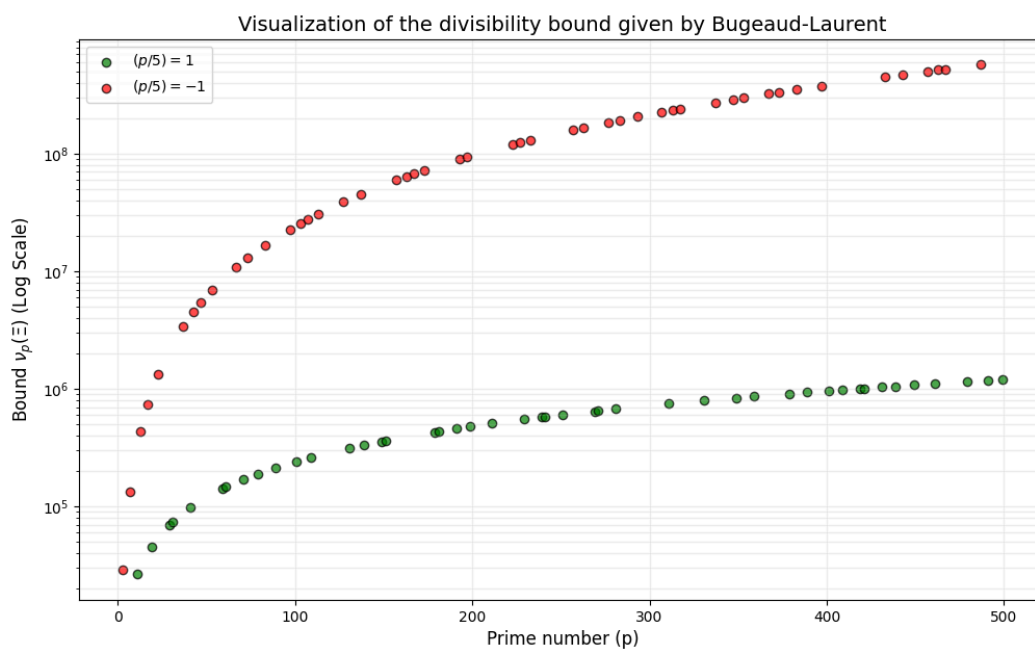


➤ **Fig 2.32.4.** The Fibonacci sequence showing $p \mid F_{p-(p|5)}$, logarithmically scaled.

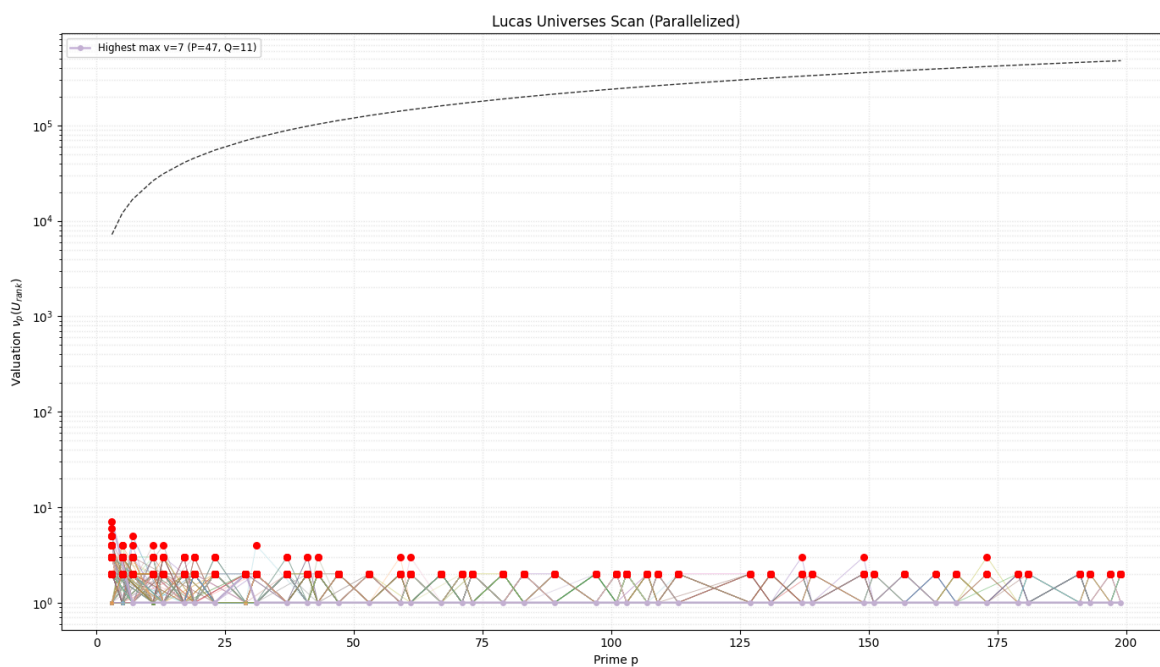
Prime p	$q(p)$	Prime p	$q(p)$	Prime p	$q(p)$
2	1/2	13	3/13	31	25/31
3	1/3	17	16/17	37	13/37
5	1/5	19	3/19	41	39/41
7	3/7	23	15/23	43	39/43
11	5/11	29	26/29	47	16/47

➤ **Data 2.43.1** Data from an HTML report from script (2.43) `wss_prime_statistical_report.py`. Here,

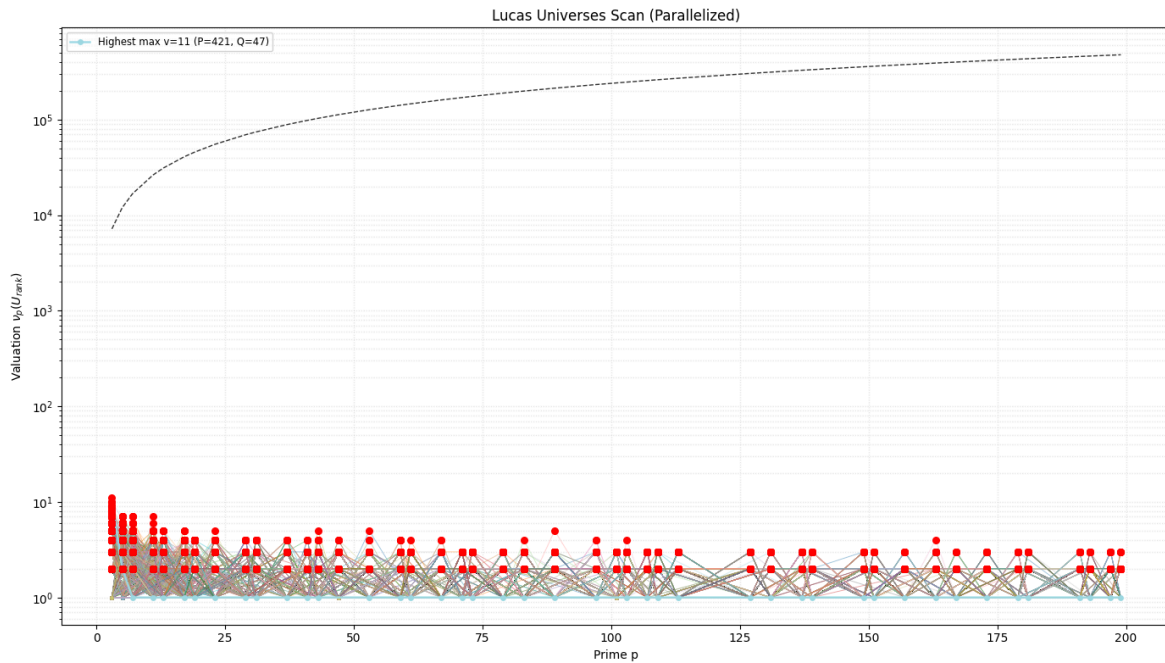
$$q(p) \equiv \frac{1}{p} F_{p-(p|5)} \pmod{p}. \text{ If } p \text{ is a WSS prime then } q(p) = 0.$$



➤ **Fig 2.33.1** A graph generated with script (2.33) `lucas_prime_bound_visualizer.py` of the prime valuation bound, split across Legendre cases.



➤ **Fig 2.35.1** A graph generated with script (2.35) `parallel_lucas_universe_overlay.py` of divisibility points from universes with $P \in [1, 50]$ and $Q \in [-25, 25]$ not including 0 or degenerate universes.



➤ **Fig 2.35.2.** A graph generated with script (2.35) `parallel_lucas_universe_overlay.py` of divisibility points from universes with $P \in [1, 750]$ and $Q \in [-375, 375]$ not including 0 or degenerate universes. Includes 562456 universes and 45 primes per universe (46 primes up to 200, and not counting primes equal to the discriminant gives the estimate of 45 per universe) for a total of $\approx 25,310,520$ data points.

Bibliography

- [1] Casselman, B. (2019). [Bruhat-Tits Tree of $SL(2)$]. Essays in harmonic analysis on p-adic $SL(2)$. <https://personal.math.ubc.ca/~cass/research/pdf/Tree.pdf>
- [2] *Welcome to Python.org*. (2026, March 10). Python.org. <https://www.python.org/>
- [3] *NumPy*. (n.d.). <https://numpy.org/>
- [4] *SciPy*. (n.d.). <https://scipy.org/>
- [5] *Matplotlib — Visualization with Python*. (n.d.). <https://matplotlib.org/>
- [6] *SymPy*. (n.d.). <https://www.sympy.org/en/>